Kadane's Algorithm :→ [Maximum Subarray Sum]

6

$O(n)$

```
int[] arr = {5, -8, 1, 2, -1, 4};   n = 6
int cmax = arr[0];   = 5
int gmax = arr[0];   = 5
for (int i = 1; i < n; i++) {
    cmax = max(arr[i], cmax + arr[i]);
    gmax = max(cmax, gmax);
}
return gmax;   (Ans)
```

i	arr[i]	Cmax	(b) cmax + arr[i]	Cmax max (a), (b)	gmax
1	-8	5	5+(-8) = -3	-3	5
2	1	-3	-3+1 = -2	1	5
3	2	1	1+2 = 3	3	5
4	-1	3	3+(-1) = 2	2	5
5	4	2	2+4 = 6	6	6

Sorting Algorithm :→ (Merge Two Sorted Arrays)

Two Pointer Approach

i=0 a = {1, 3, 5}; copy

j=0 b = {2, 4, 6, 8, 10}; paste

if a[i] < b[j] → c[k++] = a[i]++

else → b[j] < a[i] → c[k++] = b[j++]

c = {1, 2, 3, 4, 5, 6};

k = 0

0 1 2 3 4 5 6 7

k k

Merge Sort Algo :→ Divide & Conquer Algorithm (Recursion)

(log N)

split | divide

9 6 4 2 0 1 3 5

0 1 2 3 4 5 6 7

s m e

left_len = m-s+1 = 3-0+1 = 4

right_len = e-m = 7-3 = 4

(Repetitive Division)

$\frac{s+e}{2} = m$

$\frac{7}{2} = 3$

9 6 4 2 | 0 1 3 5

9 6 | 4 2 | 0 1 | 3 5

9 | 6 | 4 | 2 (Sorted) | 0 | 1 | 3 | 5 ← Single Element

6 9 | 2 4 | 0 1 | 3 5

(M2SA)

Sorted Left: 2 4 6 9

(n×log n)

Sorted Array Right: 0 1 3 5

{0 1 2 3 4 5 6 9} final sorted Array (n)

[(Merge Sort + Quick Sort)]

(Code) Monday

(Explanation + Code) Monday

n log n (pivot)

Learn & Come